# Curso AI Engineers — Naive RAG do Zero

## Ingestão → Embeddings → Recuperação → Geração

- Palestrante: Equipe AI Engineering Academy
- Progresso do Curso: Dia 2
- Objetivo Técnico: Guia prático para implementar uma arquitetura Naive RAG de ponta a ponta

# Objetivos do Curso

## Parte 1: Fundamentos de RAG e Visão Geral do Processo

**Compreensão da Arquitetura:** Desmontagem profunda do pipeline de ponta a ponta do Naive RAG.

**Ingestão e Limpeza:** Dominar o processamento e padronização de dados não estruturados.

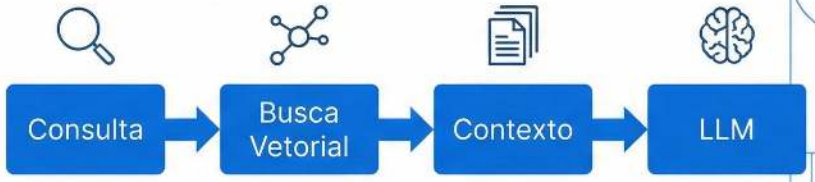
**Armazenamento Vetorial:** Construir índices de Embedding eficientes e bancos de dados vetoriais.

**Recuperação Semântica:** Implementar a recuperação de contexto baseada na similaridade Top-k.

**Geração Aumentada:** Obter respostas precisas baseadas no contexto através de engenharia de instruções.

# O que é Naive RAG?

- **Definição Básica**: A implementação mais primitiva e intuitiva da arquitetura RAG linear.
- **Mecanismo Central**: Depende totalmente da similaridade de vetores Top-k para recuperação de contexto.
- **Escopo Definido**: Não inclui reordenação (Reranking) ou transformações complexas de consulta.
- **Características da Arquitetura**: Sem lógica de agentes (Agents), com caminho de execução fixo e transparente.



# Visão Geral do Pipeline Completo (Pipeline Overview)

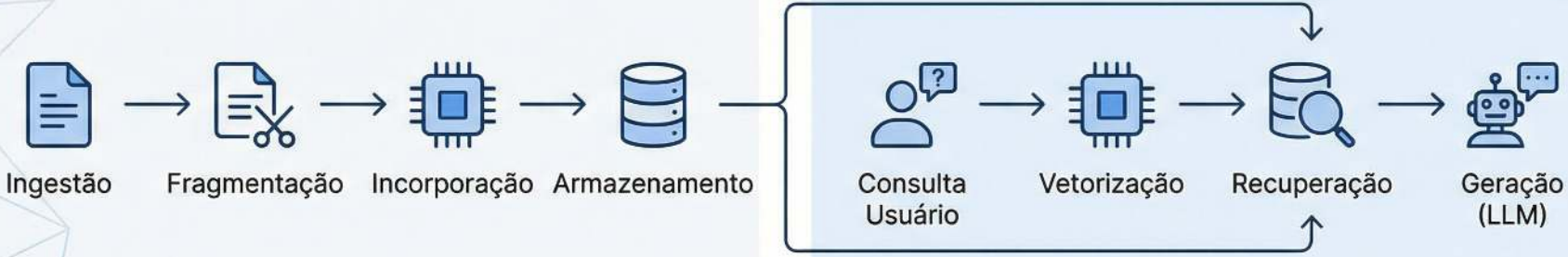
**Insight Central:** 80% dos problemas de qualidade em RAG ocorrem antes da chamada ao LLM.

## Fase Offline

Limpeza, fragmentação e vetorização de documentos, com armazenamento no banco de dados.

## Fase Online

A consulta do usuário é vetorizada, recuperada e a resposta é gerada pelo LLM.



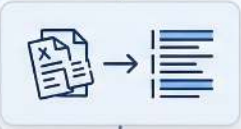

**Foco de Engenharia:** O pré-processamento de documentos e a recuperação precisa são a chave para o sucesso do sistema.

Parte 2: Aprofundamento nos Componentes Técnicos

# Ingestão de Dados

**Suporte a Múltiplas Fontes:**
Compatível com PDF, Markdown, bancos de dados e APIs internas.

**Limpeza Básica:**
Remoção de ruídos e formatação redundante para garantir a qualidade do corpus.



**Extração de Texto:**
Conversão de dados não estruturados em um fluxo de texto puro processável.

**Regra de Ouro:**
A precisão da ingestão de dados determina diretamente o limite superior da recuperação subsequente.

```python
# Exemplo básico de ingestão de texto
def load_document(file_path: str) -> str:
    with open(file_path, 'r', encoding='utf-8') as f:
        # Lê o conteúdo e realiza limpeza básica (remove espaços no início e fim)
        content = f.read().strip()
    return content

raw_text = load_document("knowledge_base.txt")
print(f"Caracteres carregados: {len(raw_text)}")
```

# Fragmentação de Documentos (Chunking)

- **Parâmetros Principais:** Tamanho do Bloco (Chunk Size) e Sobreposição (Overlap)
- **Melhores Práticas:** Recomenda-se manter entre 500–1000 Tokens
- **Riscos Potenciais:** A fragmentação inadequada causa quebra semântica e reduz a precisão da recuperação
- **Comparação de Qualidade:** Mostrando fragmentação lógica completa vs fragmentação de má qualidade com contexto truncado

**Exemplo Típico: Usando a Fragmentação Recursiva do LangChain**

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=100,
    separators=["\n\n", "\n", " ", " "]
)
chunks = splitter.split_text(raw_text)
```

**Comparação de Qualidade**

Fragmentação Lógica Completa, e.g. a full sentence da portencs causa quebra semântica e reduz a precisão da recuperação.

Fragmentação de Má qualidade, e.g. a sentenho de midward causa quebra semântica e reduz a precisão da recuperação.

→ **Redução da Precisão da Recuperação**

**Fragmentação Lógica Completa** | **Fragmentação de Má Qualidade**

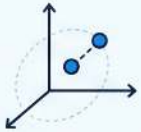
**Estratégias de Fragmentação**

| Estratégia de Fragmentação | Vantagens | Desvantagens |
|---|---|---|
| Comprimento Fixo (Fixed) | Rápido de calcular, implementação simples | Facilmente corta frases ou semântica |
| Caractere Recursivo (Recursive) | Mantém a integridade de parágrafos e frases | O tamanho do bloco pode ser irregular |
| Fragmentação Semântica (Semantic) | Consistência semântica mais alta | Alto custo computacional, lento |

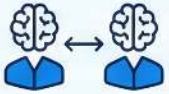
# Incorporações (Embeddings)

Segunda Parte: Aprofundamento nos Componentes Técnicos

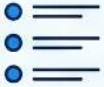
**Princípio Central:** Converte texto não estruturado em vetores matemáticos em um espaço de alta dimensão



**Captura Semântica:** Vetores próximos no espaço representam significados semelhantes, permitindo correspondência difusa



**Consistência do Modelo:** O texto da consulta e os documentos indexados devem usar exatamente o mesmo modelo de incorporação



**Prática de Engenharia:** Recomenda-se usar o modelo `text-embedding-3` da OpenAI ou modelos BGE de código aberto

Exemplo de geração de vetor de incorporação com OpenAI

```
# Exemplo usando OpenAI para gerar vetores de incorporação
from openai import OpenAI
client = OpenAI()

def get_embedding(text):
    # Certifique-se de que a versão do modelo é consistente com a indexação
    response = client.embeddings.create(
        input=[text],
        model="text-embedding-3-small"
    )
    return response.data[0].embedding
```

# Armazenamento de Vetores (Vector Store)

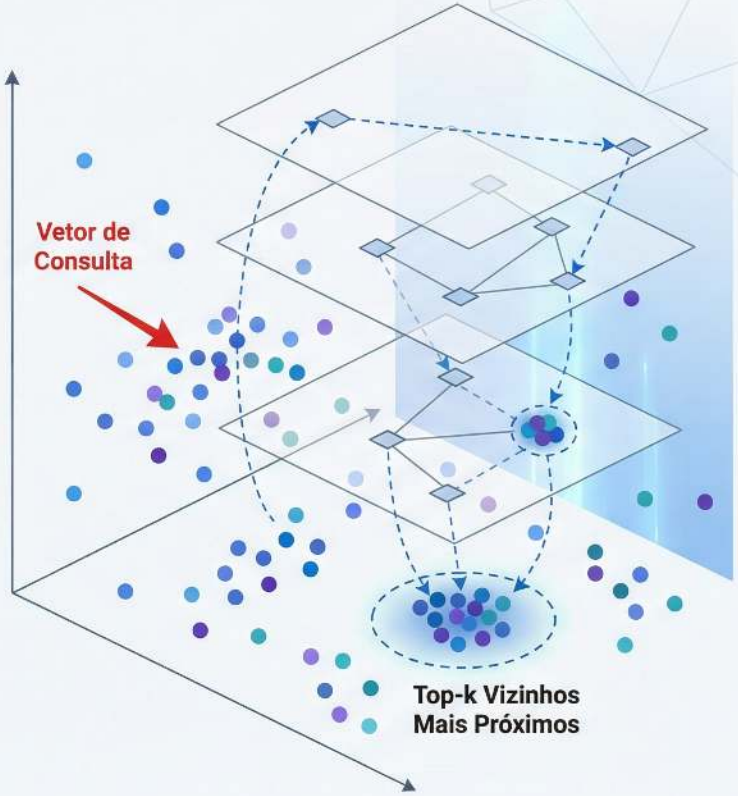
**Ecossistema de Ferramentas:** FAISS, Chroma (Local); Pinecone, Weaviate (Gerenciado)

**Mecanismo de Indexação:** Algoritmos HNSW ou IVF para recuperação rápida sublinear

**Métrica de Similaridade:** Cosseno ou Distância Euclidiana para vizinhos espaciais

**Função Principal:** Persistência e suporte a consultas simultâneas em alta escala

| Tipo de Ferramenta | Ferramentas Representativas | Vantagens Principais |
|---|---|---|
| Leve/Open Source | FAISS, Chroma | Fácil implantação, ideal para conjuntos de dados pequenos/médios e prototipagem |
| Serviço Gerenciado/Nuvem | Pinecone, Zilliz | Pronto para uso, escalabilidade horizontal massiva e alta disponibilidade |



# Mecanismo de Recuperação (Retrieval)

SEGUNDA PARTE: Aprofundamento nos Componentes Técnicos

**Similaridade de Cosseno:** Calcula a distância geométrica entre o vetor de consulta e os vetores do documento.

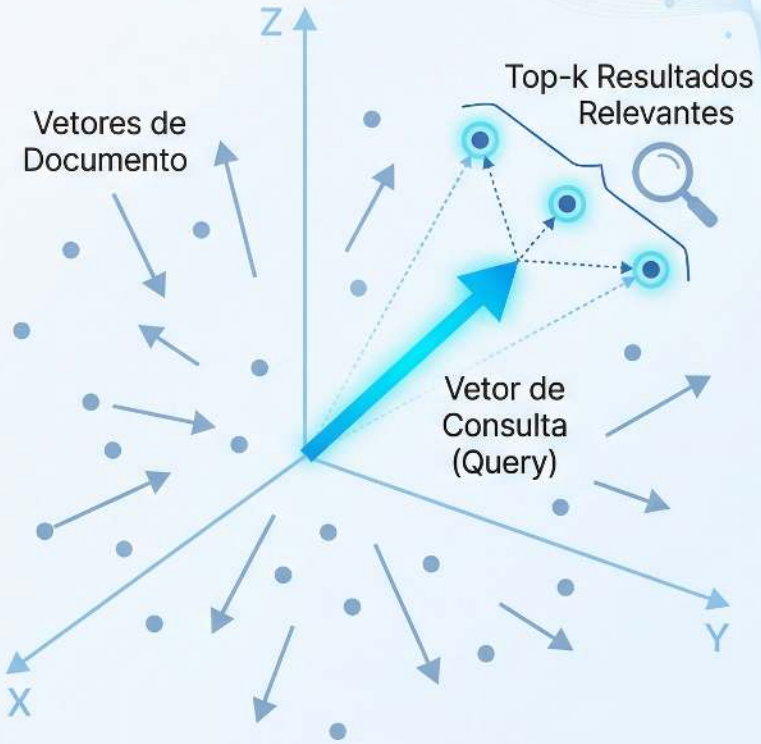
$$\frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$ Fórmula

**Estratégia Top-k:** Recupera os 3–5 fragmentos mais relevantes como contexto de referência.

**Mapeamento de Vetores:** A consulta deve usar o mesmo modelo de embedding consistente com os documentos.

**Núcleo da Qualidade:** A qualidade da entrada e a precisão da indexação determinam o efeito final da recuperação.

**Eficiência na Recuperação:** Correspondência em milissegundos através de indexação em bancos de dados vetoriais.



Parte 2: Aprofundamento nos Componentes Técnicos

# Geração de Conteúdo (Generation)

- **Modelos de Prompt:** Integração do contexto recuperado com a pergunta original do usuário.
- **Lógica Central:** O Prompt é a cola que conecta o conteúdo recuperado às capacidades do LLM.
- **Restrições Comportamentais:** Uso de System Message para garantir que o modelo responda apenas com base em fatos.
- **Estrutura Típica:** [Instrução da Tarefa] + [Contexto Recuperado] + [Consulta do Usuário]

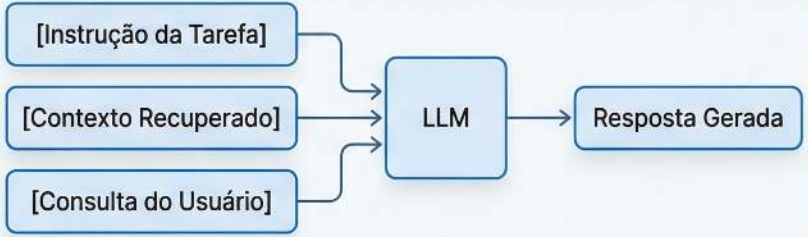
**Exemplo de Código Python**

```python
# Estrutura Típica de Prompt RAG
PROMPT_TEMPLATE = """
Use o contexto fornecido abaixo para responder à pergunta do usuário.
Se você não souber a resposta, diga que não sabe, não tente inventar.

Contexto: {context}

Pergunta: {question}

Resposta Útil:"""
```



# LIMITAÇÕES DO NAIVE RAG: DESAFIOS DO MUNDO REAL

## SEGUNDA PARTE: ANÁLISE PROFUNDA DOS COMPONENTES TÉCNICOS

### Deriva Semântica

Depende apenas da similaridade vetorial, frequentemente recuperando ruído superficialmente semelhante, mas logicamente irrelevante.

### Quebra de Chunking

Corte rígido leva à perda de contexto crucial, impedindo que o modelo obtenha a semântica completa.

### Precisão de Recuperação

Falta de mecanismo de reordenação (Reranking), resultados Top-k repletos de informações de baixa qualidade.

### Gargalo Arquitetural

Dificuldade em lidar com consultas ambíguas e raciocínio multi-salto, incapaz de lidar com lógica de negócios complexa.

### Pensamento Avançado

Avalie o cenário de negócios atual para decidir quando migrar do Naive para o Advanced RAG.

# Prática Parte 1: Persistência de Dados

## Parte 3: Prática (Hands-on)

- **Carregamento e Limpeza de Documentos**: Analisar arquivos de origem, extrair texto puro e remover ruído.
- **Estratégia de Fragmentação Semântica**: Configurar tamanho e sobreposição para garantir o contexto completo de recuperação.
- **Geração de Vetorização**: Invocar o modelo de Embedding para converter blocos de texto em vetores de alta dimensão.
- **Persistência de Índice**: Armazenar vetores e metadados em bancos de dados Chroma ou FAISS.

```python
# Exemplo de Fluxo Central de Persistência de Dados

from langchain.document_loaders import TextLoader
from langchain.text_splitter import
RecursiveCharacterTextSplitter
from langchain.vectorstores import Chroma

# 1. Carregamento e Fragmentação
loader = TextLoader("./data/manual.txt")
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
chunks = splitter.split_documents(loader.load())

# 2. Vetorização e Armazenamento
vector_db = Chroma.from_documents(
    documents=chunks,
    embedding=OpenAIEmbeddings(),
    persist_directory="./chroma_db"
)
vector_db.persist()
```

# Parte 2: Recuperação e Geração na Prática

## Parte 3: Prática (Hands-on)

**Vetorização da Consulta**
Converte a pergunta do usuário em um vetor de Embedding consistente com os documentos.

↓

**Recuperação Semântica**
Executa busca por similaridade no banco de dados vetorial, extraindo os Top-k blocos relevantes.

↓

**Montagem do Prompt**
Combina o contexto recuperado com a pergunta original para construir um Prompt aprimorado.

↓

**Chamada do Modelo**
Envia o prompt final gerado para o LLM e obtém uma resposta estruturada.

```python
# Lógica Central de Recuperação e Geração
query_embedding = embedding_model.embed(query)
docs = vector_db.similarity_search_by_vector(query_embedding, k=3)

context = "\n".join([d.page_content for d in docs])
prompt = f"Context: {context}\n\nQuestion: {query}"

response = llm.generate(prompt)
print(f"AI: {response}")
```

# Implementação de Código Completo (Full RAG)

**Engenharia Minimalista:** Apenas ~40 linhas de Python para um ciclo completo de ponta a ponta.

**Módulos Principais:** Encapsula as três funções principais: ingest, retrieve e generate.

**Lógica do Fluxo:** Cobre o link completo, da persistência de dados à geração de contexto.

**Foco Prático:** Atenção em como unir o conteúdo recuperado e a pergunta através do Prompt.

Código Fonte Python

```python
import chromadb
from openai import OpenAI

# 1. Ingest: Persistência de Documentos
def ingest(text, collection):
    collection.add(documents=[text], ids=["doc_01"])

# 2. Retrieve: Recuperação Semântica
def retrieve(query, collection):
    results = collection.query(query_texts=[query], n_results=3)
    return "\n".join(results['documents'][0])

# 3. Generate: Geração Aumentada
def generate(query, context):
    client = OpenAI()
    prompt = f"根据上下文回答: \n{context}\n\n问题: {query}"
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content

# Executar fluxo completo
# ingest(raw_text, col)
# context = retrieve("Como implementar RAG?", col)
# print(generate("Como implementar RAG?", context))
```

# Pontos-chave (Takeaways)

### Recuperação acima do Modelo

A qualidade do contexto recuperado é mais importante do que a escolha do modelo de linguagem (LLM).

### Chunking é Engenharia

A divisão de documentos (Chunking) é o desafio de engenharia mais central no RAG.

### Embeddings definem o teto

A eficácia do modelo de Embedding determina diretamente o limite máximo da qualidade da recuperação semântica.

### Lógica de Evolução

O **RAG Básico** resolve 70% dos cenários fundamentais; os 30% restantes exigem otimização avançada.

### Prática Primeiro

Em vez de otimizar excessivamente o **Prompt**, concentre-se primeiro em melhorar a taxa de recuperação por meio de engenharia.